The algorithm can also be varied by passing different constants to `password_hash()`, call `password_algos()` to see the available algorithms for your installation:

```
print_r(password_algos());
```

```
Output:
Array
(
    [0] => 2y
    [1] => argon2i
    [2] => argon2id
)
```

Most PHP installations will have the above by default, and PHP even defines core constants that map to those strings: `PASSWORD_BCRYPT` (which is also `PASSWORD_DEFAULT`), `PASSWORD_ARGON2I`, and `PASSWORD_ARGON2ID` map to `2y`, `argon2i`, and `argon2id`, respectively. Remember that the hashes produced by `password_hash()` contain enough information about the algorithm used to verify a string, no extra arguments are required:

```
var_dump(password_verify(
    'weaksauce',
    password_hash("weaksauce", PASSWORD_DEFAULT)
));
var_dump(password_verify(
    'weaksauce',
    password_hash("weaksauce", PASSWORD_ARGON2I)
));
```

```
Output:
bool(true)
bool(true)
```

One other aspect of using `password_verify()` is that it takes a fixed length of time to execute. This is important because if the verifying code varies in the length of time it takes to reject a password, this opens up a vulnerability. By measuring variances in execution time, it would be possible to home in on the correct password (older versions of Windows had this problem). Exploits targeting such vulnerabilities – whereby an adjacent metric such as time, or variance in power consumption, can be used to gain useful information about encrypted data – are known as *side-channel* attacks.